DUALE HOCHSCHULE SACHSEN

Dr. M. Hofmann

Verteilte Systeme

Wintersemester 2026

4. Übungsserie

Aufgabe 1

Machen Sie sich mit dem Google Remote Procedure Call (gRPC) Framework für Python vertraut, indem Sie das nachfolgende Beispiel ausführen.
Hinweis: Installieren Sie gRPC für Python mit den Anweisungen am Ende dieses Übungsblattes.
Datei: `hello.proto`

```
syntax = "proto3";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

message HelloRequest {
  string name = 1;
}

message HelloReply {
  string message = 1;
}
```

Kommando:

```
#!/bin/sh
rm -rf generated ; mkdir generated

python3 -m grpc_tools.protoc \
    -I. --python_out=generated --grpc_python_out=generated \
    hello.proto
```

Datei: `server.py`

```python
import sys
sys.path.append("generated")

from concurrent import futures
import grpc
import hello_pb2
import hello_pb2_grpc

class Greeter(hello_pb2_grpc.GreeterServicer):
    def SayHello(self, request, context):
        return hello_pb2.HelloReply(message=f"Hello, {request.name}!")

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    hello_pb2_grpc.add_GreeterServicer_to_server(Greeter(), server)
    server.add_insecure_port("[::]:50051")
    server.start()
    server.wait_for_termination()

if __name__ == "__main__":
    serve()
```

Datei: `client.py`

```python
import sys
sys.path.append("generated")

import grpc
import hello_pb2
import hello_pb2_grpc

def run():
    with grpc.insecure_channel("localhost:50051") as channel:
        stub = hello_pb2_grpc.GreeterStub(channel)
        response = stub.SayHello(hello_pb2.HelloRequest(name="Google"))
        print("Greeter client received: " + response.message)

if __name__ == "__main__":
    run()
```

Aufgabe 2

Erweitern Sie das bestehende gRPC-Beispiel, so dass die übertragenen Nachrichten wie folgt erweitert werden:

a) Übertragung von Feldern mit Basisdatentypen Integer, Float, Bool.

b) Übertragung von Feldern mit Listen von Basisdatentypen.

c) Übertragung eines Feldes mit Aufzählungsdatentyp (d. h. `enum`).

d) Übertragung eines Feldes mit einer Klassendatenstruktur.

e) Übertragung eines Feldes welches alle vorhergehenden Datentypen enthält, z. B. Liste von Klassendatenstrukturen mit Feldern von Basis- und Aufzählungsdatentypen.

Aufgabe 3

Machen Sie sich mit der Nutzung des gRPC-Frameworks in einem C++-Programm vertraut.

a) Implementieren und testen Sie den gRPC-Client und Server mit C++ anhand des nachfolgenden Beispiels.

b) Passen sie das Beispiel so an, dass die gleichen Datentypen für die Nachrichtenübertragung wie in der vorhergehenden Aufgabe benutzt werden (d. h. Basis- und Aufzählungsdatentypen, Listen, Klassen, . . .).

c) Testen Sie die Interoperabilität zwischen den Python- und C++-Programmen indem Sie Python-Client und C++-Server bzw. C++-Client und Python-Server miteinander ausführen.

Hinweis: Installieren Sie gRPC für Python mit den Anweisungen am Ende dieses Übungsblattes.
Hinweis: Falls Sie nicht (ausreichend) mit C++ vertraut sind, versuchen Sie ein alternative, von gRPC unterstützte Programmiersprache für die Aufgabe zu verwenden.
Datei: `hello.proto`

```
syntax = "proto3";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

message HelloRequest {
  string name = 1;
}

message HelloReply {
  string message = 1;
}
```

Datei: `server.cpp`

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <grpcpp/grpcpp.h>
#include "hello.grpc.pb.h"

using grpc::Server;
using grpc::ServerBuilder;
using grpc::ServerContext;
using grpc::Status;

class GreeterServiceImpl final : public Greeter::Service {
  Status SayHello(ServerContext* context, const HelloRequest* request,
                  HelloReply* reply) override {
    std::string prefix("Hello ");
    reply->set_message(prefix + request->name());
    return Status::OK;
  }
};

void RunServer() {
  std::string server_address("0.0.0.0:50051");
  GreeterServiceImpl service;
  ServerBuilder builder;
  builder.AddListeningPort(server_address, grpc::InsecureServerCredentials());
  builder.RegisterService(&service);
  std::unique_ptr<Server> server(builder.BuildAndStart());
  std::cout << "Server listening on " << server_address << std::endl;
  server->Wait();
}

int main() {
  RunServer();
  return 0;
}
```

Datei: `client.cpp`

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <grpcpp/grpcpp.h>
#include "hello.grpc.pb.h"

using grpc::Channel;
using grpc::ClientContext;
using grpc::Status;

class GreeterClient {
 public:
  GreeterClient(std::shared_ptr<Channel> channel)
    : stub_(Greeter::NewStub(channel)) {}

  std::string SayHello(const std::string& user) {
    HelloRequest request;
    request.set_name(user);
    HelloReply reply;
    ClientContext context;

    Status status = stub_->SayHello(&context, request, &reply);

    if (status.ok()) {
      return reply.message();
    } else {
      std::cout << status.error_code() << ": " << status.error_message()
        << std::endl;
      return "RPC failed";
    }
  }

 private:
  std::unique_ptr<Greeter::Stub> stub_;
};

int main(int argc, char** argv) {
  GreeterClient greeter(grpc::CreateChannel("localhost:50051",
    grpc::InsecureChannelCredentials()));
  std::string user("Google");
  std::string reply = greeter.SayHello(user);
  std::cout << "Greeter received: " << reply << std::endl;
  return 0;
}
```

Kommando:

```
rm -rf generated_cpp ; mkdir generated_cpp

protoc -I. \
    --grpc_out=generated_cpp --plugin=protoc-gen-grpc=`which grpc_cpp_plugin` \
    hello.proto

protoc -I. \
    --cpp_out=generated_cpp \
    hello.proto

c++ -Igenerated_cpp -o client \
    client.cpp generated_cpp/hello.grpc.pb.cc generated_cpp/hello.pb.cc \
    -lgrpc++ -lprotobuf -lgpr -labsl_synchronization -lpthread -ldl -lz

c++ -Igenerated_cpp -o server \
    server.cpp generated_cpp/hello.grpc.pb.cc generated_cpp/hello.pb.cc \
    -lgrpc++ -lprotobuf -lgpr -labsl_synchronization -lpthread -ldl -lz
```

Installation von gRPC für Python unter Windows

1. Eingabeaufforderung/Terminal (`cmd`) im Arbeitsverzeichnis öffnen.
2. Erstellen einer virtuellen Python-Umgebung: `python -m venv .venv`
3. Virtuelle Python-Umgebung aktivieren: `.venv\Scripts\activate.bat`
4. Python-Abhängigkeiten installieren: `pip install grpcio-tools`

Installation von gRPC für Python unter Ubuntu 24.04 Linux

1. Eingabeaufforderung/Terminal (`bash`) im Arbeitsverzeichnis öffnen.
2. Erstellen einer virtuellen Python-Umgebung: `python3 -m venv .venv`
3. Virtuelle Python-Umgebung aktivieren: `.venv/bin/activate`
4. Python-Abhängigkeiten installieren: `pip install grpcio-tools`

Installation von gRPC für C++ unter Ubuntu 24.04 Linux

1. Eingabeaufforderung/Terminal (`bash`) im Arbeitsverzeichnis öffnen.
2. Abhängigkeiten (u. a. C++- und Protobuf-Compiler, ...) installieren:
   `sudo apt install protobuf-compiler protobuf-compiler-grpc libgrpc++-dev`